I do not like computer jokes …
not one bit!

CMPT 295

Unit - Instruction Set Architecture

Lecture 25 – ISA Design

Last Lecture

- **`Assembler`** (part of the compilation process):
  - Transforms assembly code (`movl %edi, %eax`) into machine code (`0xf889 -> 1111100010001001`)
  - Machine code -> series of 0's and 1's
    - Control the microprocessor h/w
- Instruction Set Architecture (ISA)
  - **Definition**: Interface between the hardware and the software, specifying both what the microprocessor is capable of doing as well as how it gets done.
  - **Defines**:
    1. Memory model
    2. Registers
    3. Set of instructions (assembly-machine)
    4. Model of computation

    etc...

Today's Menu

- Instruction Set Architecture (ISA)
  - Definition of ISA
- Instruction Set design
  - Design guidelines
  - Example of an instruction set: MIPS
  - Create our own instruction sets
  - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
  - Execution of machine instructions (datapath)
  - Intro to logic design + Combinational logic + Sequential logic circuit
  - Sequential execution of machine instructions
  - Pipelined execution of machine instructions + Hazards

3 common Types of **Instruction Set Architecture**

1. ***Stack*** – The operands are implicit and are pushed onto the stack.
2. ***Accumulator*** – One operand is implicit, it is the register called ***accumulator***.
3. ***General Purpose Register (GPR)*** – All operands are explicit. They are either registers or memory addresses.

Example:

Here is a C statement -> **`C = A + B;`**
Compiled into these three **ISA**s we get:

Assuming **`A`** is in **`%edi`**, **`B`** is in **`%esi`** and **`C`** is returned in **`%eax`**

Stack	Accumulator	GPR	
PUSH A	LOAD A	LOAD R1,A	`movl %edi, %edx`
PUSH B	ADD B	ADD R1,B	`addl %esi, %edx`
ADD	STORE C	STORE R1,C	`movl %edx, %eax`
POP C	-	-	

Source: http://www.cs.kent.edu/~durand/CS0/Notes/Chapter05/isa.html

Types of **Instruction Set**

CISC

- **C**omplex **I**nstruction **S**et **C**omputer
- Large # of instructions including special purpose instructions
- Usually "**register-memory**" architecture
  - This means that any instructions can access memory
- Examples: VAX, **x86**, MC68000

RISC

- **R**educed **I**nstruction **S**et **C**omputer
- Small # of general purpose instructions + all instructions have the same size
  - therefore, simpler microprocessor design
- "**load/store**" architecture
- Examples: SPARC, **MIPS**, Alpha AXP, PowerPC
- **C = A + B**; assembled into:

```
LOAD  R1, A
LOAD  R2, B
ADD   R3, R1, R2
STORE C, R3
```

Instruction set (**IS**) design principles

What this principle means to us **IS** designers is that we must assign a unique **opcode** to each instruction

1. Each instruction of **IS** must have an unambiguous **binary encoding** so microprocessor can unambiguously decode and execute it
2. **IS** must be functionally complete, i.e., must be "Turing complete"
    1. Data transfer instructions — Storage reference
    2. Data manipulation instructions — Arithmetic and logical
    3. Program control instructions — Branch and jump

    3 classes of instructions
3. In terms of ***machine code instruction format***:
    a. **Ideally**, the machine code instructions in this **IS** must all have the **same length** and use the **same format!**
    b. If we cannot achieve **a.**, we must create as few formats as possible
    c. If we cannot achieve **a.**, then let's try to position the **fields** of our **machine code instruction formats** that have the same **purpose** at the **same location** in the **machine code instruction formats**

Let's expand!

1. "Each instruction of **IS** must have an unambiguous **binary encoding** …"

As part of the **IS** we have …

Assembly instructions compile *(more specifically: "assemble")* into **Machine code instructions**

- An assembly instruction is a **symbolic representation** of a machine instruction
  - **Mnemonics**: abbreviation of operation name
    - Example: `movq`, `mulw`, `ret`
  - **Labels** to represent addresses
    - Example:
```
call sum
jmp loop
```
- **Advantage**: human readable, i.e., program easier to read and write than a series of 0's and 1's,
- Made easier through the use of mnemonics and labels

- Each assembly instruction has a corresponding machine code instruction
- A machine code instruction is expressed as bit pattern (**binary encoding**) –> 0's and 1's

unique bit pattern representing each **opcode**

unique bit pattern representing each **operand**

Example of a common machine code instruction **format**

What is an **opcode**? What is an **operand**?

- **Opcode**: **Op**eration **code**
- **Opcode**: Operation that can be executed by the microprocessor
  - Expressed as bit pattern (**binary encoding**) –> 0's and 1's
- **Operand(s)**: required by the opcode in order for microprocessor to successfully carry out the instruction
  - They are also expressed as bit patterns –> 0's and 1's
- In the output of the **`objdump`** tool (disassembler), we can see opcodes and operands expressed as hexadecimal values

Example using x86-64

```
00000000004004b7 <someFcn>:
  4004b7:   4c 03 00      add     (%rax),%r8
  4004ba:   7e fb         jle     4004b7 <someFcn>
  4004bc:   c3            retq
```

```
000000000000001101001100
1111101101111110
11000011
```

Summary

- 3 common Types of **Instruction Set Architecture**
  1. *Stack*
  2. *Accumulator*
  3. *General Purpose Register (GPR)*
- Types of **Instruction Set**: CISC and RISC
- Instruction set (**IS**) design principles

Next lecture

- Instruction Set Architecture (ISA)
  - Definition of ISA
- Instruction Set design
  - Design guidelines
  - Example of an instruction set: MIPS
  - Create our own instruction sets
  - ISA evaluation
- Implementation of a microprocessor (CPU) based on an ISA
  - Execution of machine instructions (datapath)
  - Intro to logic design + Combinational logic + Sequential logic circuit
  - Sequential execution of machine instructions
  - Pipelined execution of machine instructions + Hazards